

软件过程改进方案设计文档

1. 概述

1.1 文档目的

本文档基于《AI技术应用场景匹配报告》的匹配分析结果，针对HarmonyWardrobe项目的软件过程，设计AI技术与传统过程的融合改进方案。重点覆盖软件实现过程、软件测试过程和需求工程过程三个核心改进方向，包含过程重构设计、工具链选型、集成方案与风险评估。

1.2 改进范围

本次改进覆盖三个过程。需求工程过程的改进方向为AI辅助需求文档自动生成与质量校验，优先级为P1。软件实现过程的改进方向为AI代码生成辅助和AI智能代码审查，优先级为P0。软件测试过程的改进方向为AI测试用例生成和AI缺陷预测分析，优先级为P0。

2. 过程重构设计

2.1 新增AI相关角色

为适配AI技术的引入，在现有团队角色基础上新增三个角色。

第一个是AI训练师，其核心职责包括设计并优化各环节AI工具的Prompt模板、维护项目级上下文知识库（涵盖代码库、设计文档和历史用例），以及持续跟踪AI输出质量并迭代Prompt策略。在课程项目的规模下，该角色可由一名开发工程师兼任，参与需求工程、软件实现和软件测试全流程。

第二个是AI测试分析师，其核心职责包括配置AI测试生成工具参数、审核AI生成的测试用例质量、管理缺陷预测模型以及跟踪AI测试覆盖率指标。该角色可由测试工程师兼任，主要在软件测试过程中发挥作用。

第三个是AI输出审核员，其核心职责是对AI生成的所有输出物（包括代码、文档和用例）进行质量审核与验证，确保AI输出符合项目质量标准，并对AI输出中的幻觉或错误进行标记和修正。该角色由各阶段负责人兼任——需求阶段由需求分析师承担，编码阶段由开发工程师承担，测试阶段由测试工程师承担，覆盖全流程。

2.2 各过程调整方案

2.2.1 需求工程过程改进

改进前的需求工程流程为：用户需求经过人工撰写需求文档，然后进行人工评审，最终形成需求基线。改进后的流程为：用户需求先由AI辅助生成需求文档，经过人工审核优化，再由AI进行一致性校验，最后进行人工评审后形成需求基线。

具体调整包括三项。第一，新增AI辅助需求文档生成节点，使用大语言模型将用户故事或对话纪要自动转化为结构化的需求文档。第二，新增AI一致性校验节点，自动检测需求之间的冲突、冗余、遗漏以及可测试性问题。第三，需求评审会议前置AI预处理，从而缩减人工评审的轮次。

关于AI输出物的质量标准，需求条目覆盖率应不低于95%（以用户原始需求为基准），需求冲突检测率应不低于90%，每千字需求文本的错误率应不超过3处（经人工修正后）。

2.2.2 软件实现过程改进

改进前的软件实现流程为：设计方案经过人工编码实现，然后进行人工代码审查，再进行单元测试和集成。改进后的流程为：设计方案先由AI辅助代码生成，经过人工审核优化，然后由AI进行代码审查，通过后再进行人工复核，最后进行单元测试和集成。

具体调整包括四项。第一，新增AI代码生成节点，开发者使用Cursor或GitHub Copilot等工具生成代码骨架与核心业务逻辑。第二，新增AI代码审查节点，使用CodeRabbit等工具自动进行逐行代码审查。第三，AI审查作为代码门禁，通过后才进入人工复核阶段。第四，开发者的角色从代码编写者转型为代码审核与架构决策者。

关于AI输出物的质量标准，AI生成代码需通过静态分析且零高危漏洞，代码风格需100%匹配项目的eslint或prettier配置，AI代码审查的覆盖率应不低于90%的修改文件。开发者不得在未审核的情况下合并AI生成的代码。

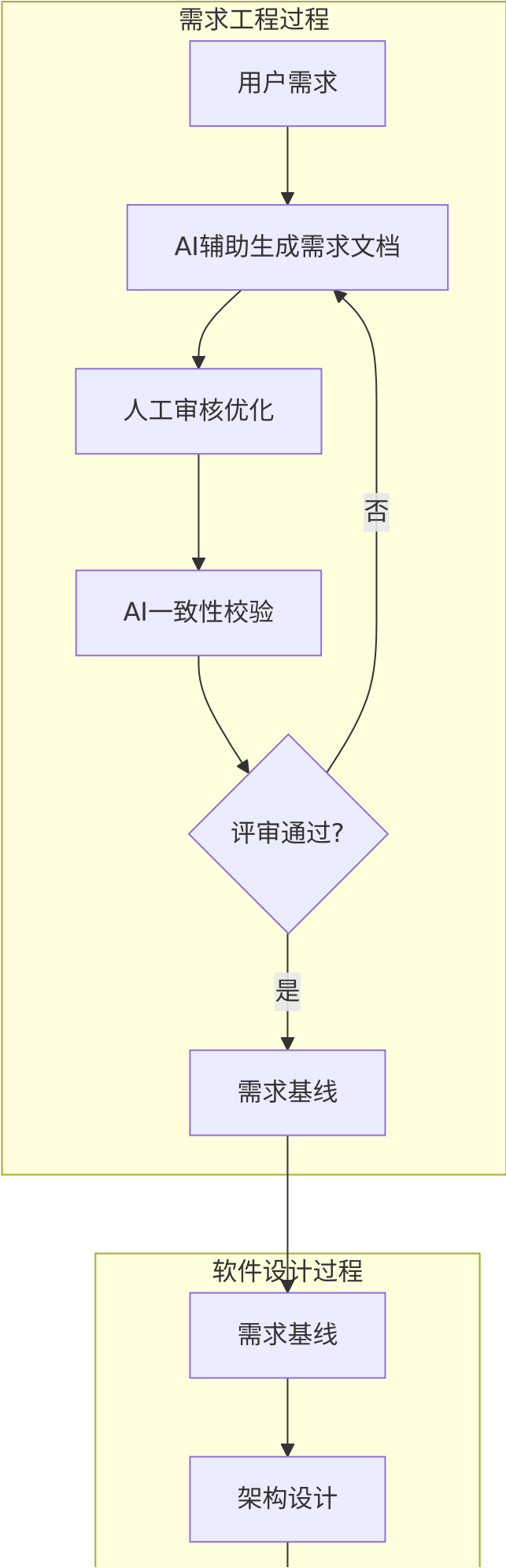
2.2.3 软件测试过程改进

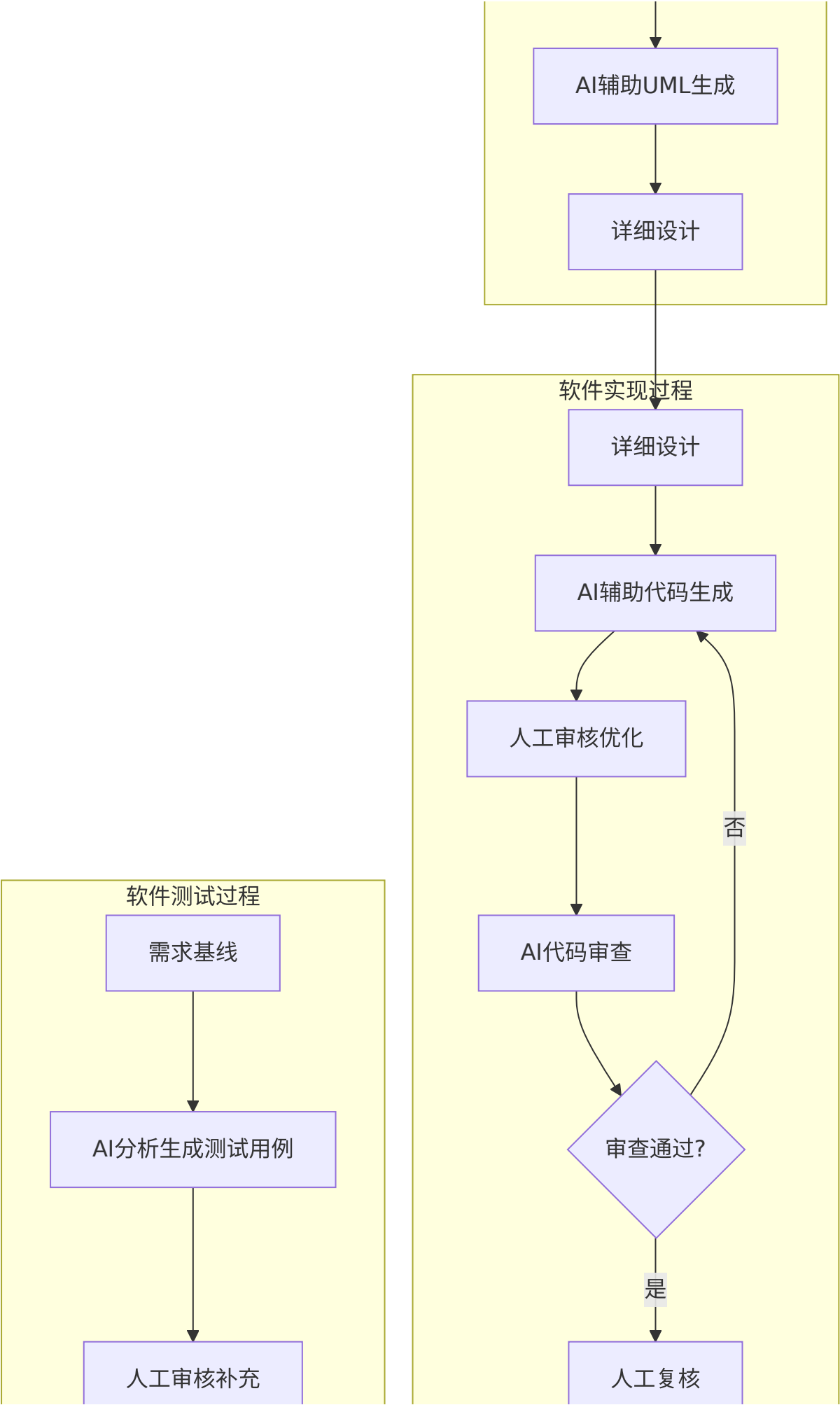
改进前的软件测试流程为：测试计划经过人工设计测试用例，然后人工编写测试脚本，再执行测试并进行缺陷跟踪。改进后的流程为：测试计划先由AI分析需求生成测试用例，经过人工审核补充，再由AI生成测试脚本，然后执行测试（引入自愈式机制），再进行AI缺陷分析，最后进行缺陷跟踪。

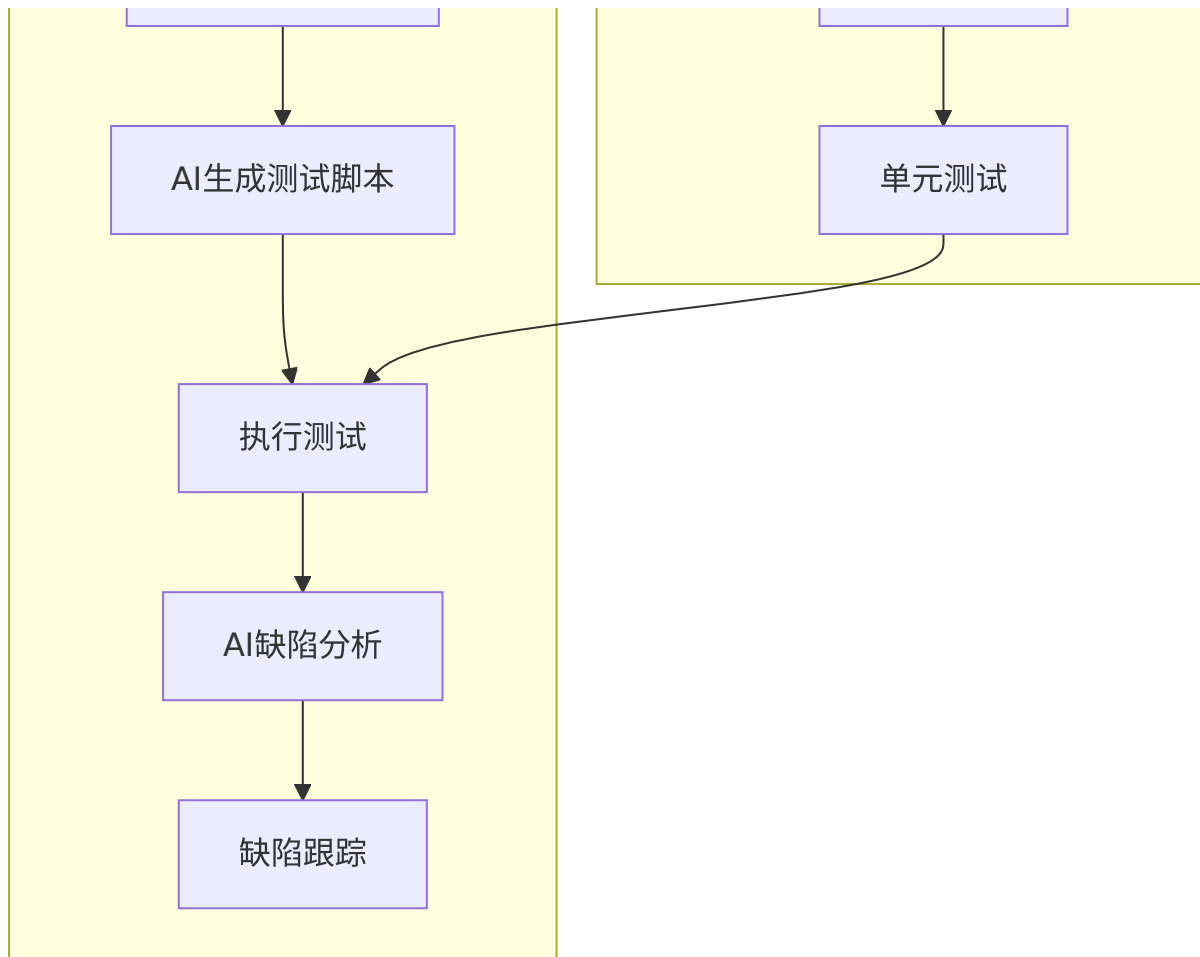
具体调整包括四项。第一，新增AI分析需求生成测试用例节点，基于需求文档自动生成功能测试用例，包含边界值和异常场景。第二，新增AI生成测试脚本节点，将测试用例自动转化为可执行的测试脚本。第三，引入自愈式测试机制，AI自动修复因UI变化导致的脚本失效。第四，新增AI缺陷分析节点，AI自动对测试失败进行根因分析并生成缺陷报告。

关于AI输出物的质量标准，测试用例对需求的覆盖率应不低于95%，AI生成的测试脚本首次执行成功率应不低于80%，缺陷报告的准确率应不低于85%（经人工验证后）。

2.3 整体过程融合流程图







3. 工具链选型

3.1 工具选型方案

在需求文档生成环节，推荐使用ChatGPT或Claude，其核心功能是将自然语言转化为结构化需求文档并进行一致性检查，可通过API或Web端集成，预期可使需求编写时间缩短50%至60%。

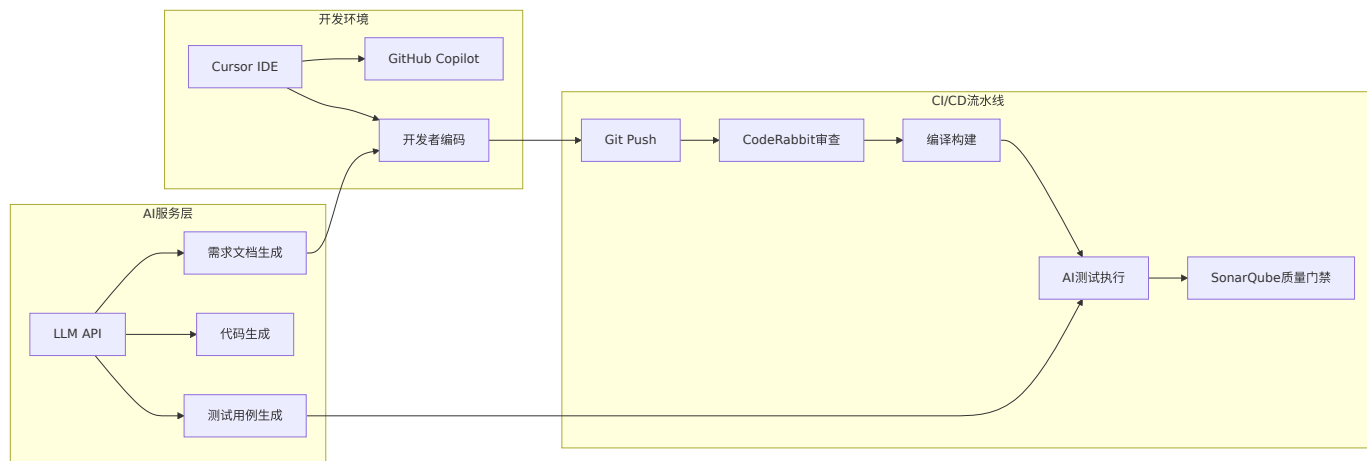
在AI代码生成环节，推荐使用Cursor或GitHub Copilot，其核心功能包括代码补全、多文件编辑和Agent模式，通过IDE插件集成，预期可使编码效率提升30%至40%。

在AI代码审查环节，推荐使用CodeRabbit，其核心功能是进行逐行PR审查并集成40余种linter，通过GitHub App集成，预期可使审查时间缩短40%至50%。

在AI测试用例生成环节，推荐使用Diffblue或CoTester，其核心功能是自动生成测试用例并集成到CI流水线中，通过CLI或CI插件集成，预期可使用例编写效率提升400%。

在AI缺陷分析环节，推荐使用SonarQube AI增强版配合ChatGPT，其核心功能是进行静态分析的AI增强和根因分析，通过CI集成，预期可使缺陷定位时间缩短40%。

3.2 工具集成架构



4. 风险评估与应对策略

4.1 风险评估矩阵

在数据安全方面，AI工具在处理项目代码和需求时可能存在信息泄露的风险，影响等级为高，发生概率为中，综合风险等级为高。在模型偏见方面，AI生成的测试用例或代码可能存在偏向性，导致遗漏特定场景，影响等级为中，发生概率为中，综合风险等级为中。在技术依赖方面，过度依赖AI工具可能导致团队核心能力退化，影响等级为高，发生概率为中，综合风险等级为高。在输出质量方面，AI生成物可能存在幻觉（Hallucination）问题，引入隐蔽缺陷，影响等级为高，发生概率为高，综合风险等级为极高。在工具兼容性方面，AI工具与HarmonyOS技术栈的兼容性存在不确定性，影响等级为中，发生概率为低，综合风险等级为中。

4.2 应对策略

针对数据安全风险，可以采用本地部署的AI模型来处理敏感代码，例如使用Ollama配合CodeLLaMA。云端AI工具仅用于通用性任务，如需求文档和测试用例的生成，不传输完整的项目代码。同时与AI工具服务商签署数据不用于训练的协议，确保数据隐私。

针对模型偏见风险，AI生成的内容必须经过人工审核并补充边界条件。建立AI输出多样性检查机制，要求AI生成多组方案供选择。同时保留传统的测试用例设计方法作为补充，确保覆盖的全面性。

针对技术依赖风险，实行AI辅助、人工决策的基本原则，禁止直接合并未经审核的AI输出。定期组织无AI工具的编码练习，保持团队核心编码能力。建立AI工具使用规范文档，明确哪些环节可以使用AI、哪些环节不可使用AI。

针对AI输出质量风险（这是最核心的风险），实施四项措施。第一是双重验证机制，所有AI生成物需通过AI自检加人工审核两道关卡。第二是建立AI输出基线，记录每次AI输出的准确率，持续监控质量趋势。第三是由AI训练师持续优化Prompt模板，通过针对性Prompt优化减少幻觉的发生。第四是渐进式信任策略，初始阶段100%人工审核，根据AI的表现逐步调整审核粒度。

针对工具兼容性风险，优先选择支持HarmonyOS和ArkTS开发的AI工具，在正式采用前进行POC验证，同时准备传统方案作为备选回退路径。

5. 预期效益汇总

指标	改进前估计	改进后估计	提升幅度
需求文档编写周期	5天	2至3天	40%至60%
编码阶段人日投入	30人日	18至21人日	30%至40%
代码审查周期	3天	1.5天	50%
测试用例设计周期	5天	1至2天	60%至80%
缺陷检测率（发布前）	75%	90%	+15个百分点
回归测试执行时间	2天	0.5天	75%

参考文献

[1] Deloitte, "The Inflection Point: Agentic AI in Software Engineering — From SDLC to AO-DLC," May 2026.

[2] H. Cheng et al., "Generative AI for Requirements Engineering: A Systematic Literature Review," *Software: Practice and Experience*, Nov. 2025.

[3] AI for Coding & Software Development, AIBuzz, May 2026.

[4] J. Schiesel, "Beyond Code Generation: How AI Is Expanding Across the Entire SDLC," Metacto, Apr. 2026.

[5] T. Patel, "AI Test Case Generation with CoTester: Smarter QA Automation," TestGrid Blog, Oct. 2025.

[6] A. Seelamneni, "AI in QA: Transforming Test Automation and Software Quality Through Intelligent Solutions," *WJAETS*, 2025.